# GeoApp SSO

## Complete Developer Implementation & Integration Guide

OIDC Provider • Portal • Web Applications • Mobile Applications • Provisioning

| | |
|---|---|
| **Document purpose** | Future-reference developer guide for the GeoApp identity platform and child applications. |
| **Security posture** | Secrets intentionally redacted from this document. A committed-secret remediation note is included. |

# Table of Contents

1. Executive Summary
2. Repository & System Inventory
3. Architecture Overview
4. Authentication Models Used in GeoApp
5. Core OIDC Provider Implementation (sso-backend)
6. Portal Login to Silent SSO Session
7. Application Registration Model
8. Credentials: What You Receive, Where They Go, and Why
9. Web Application Integration - TeamField
10. TeamField Frontend Session Handling
11. Mobile Application Integration - Ionic React
12. User Access, Roles, and Application Launcher
13. Automatic User Provisioning
14. Logout and Session Revocation
15. Configuration & Environment Variables
16. New Application Onboarding Runbook
17. End-to-End Request Flows
18. Troubleshooting Guide
19. Security Review & Mandatory Remediation
20. Recommended Standardization Plan
21. Developer Checklist
22. Source Reference Map
23. Appendix: Example Registration Payloads

> **Reading the document**
> Sections 5-14 describe the current implementation as observed in code. Sections 16 and 20 turn that implementation into a repeatable integration standard for future applications. Items that could not be independently verified are explicitly marked.

# 1. Executive Summary

GeoApp SSO is a centralized identity platform built around an OIDC provider on auth.geoapp.in / api.auth.geoapp.in. The supplied codebase combines two related but distinct authentication layers: a portal/session system used by the GeoApp login experience, and an OIDC authorization-code system used by child applications. The critical bridge is the shared OIDC session created after a successful portal login, which lets a user launch a first-party application without entering the password again.

- The SSO backend is Node.js/TypeScript with PostgreSQL, TypeORM, Redis, BullMQ, and oidc-provider.
- The main client portal is a React SPA that authenticates against the SSO backend using JWT access/refresh tokens and also relies on the browser-held OIDC session cookie for app launches.
- The admin application manages application registrations, client secrets, redirect URIs, scopes, roles, access assignments, provisioning settings, and operational retries.
- TeamField is a relying party (child application). Its ASP.NET backend owns the OIDC client secret and performs the code/state handling; the React frontend receives a short-lived one-time ticket and exchanges that ticket for a TeamField JWT.
- The TeamField mobile app uses a deep-link based authorization response (`teamfield://sso/callback`) and then calls the TeamField backend for ticket exchange; the mobile app does not need the OIDC client secret.
- User-to-application access is explicit in the SSO database. The same access assignment can trigger asynchronous provisioning into the child application, protected with a separate server-to-server provisioning secret.
- The platform stores OIDC client secrets and provisioning secrets encrypted at rest. Provisioning uses AES-256-GCM; the OIDC provider requires signing keys in a production JWKS file.
- A serious security action is required immediately: TeamField `appsettings.json` contains production-looking credentials for JWT, SMTP, AWS, and OIDC. These values must be rotated and moved into a secret store; this guide never reproduces them.

> **Core mental model**
> Think of GeoApp SSO as: (1) GeoApp is the Identity Provider, (2) TeamField is a Relying Party / OIDC client, (3) the TeamField frontend is not the holder of the confidential OIDC secret, and (4) the local TeamField JWT is a child-application session token, not the GeoApp OIDC token.

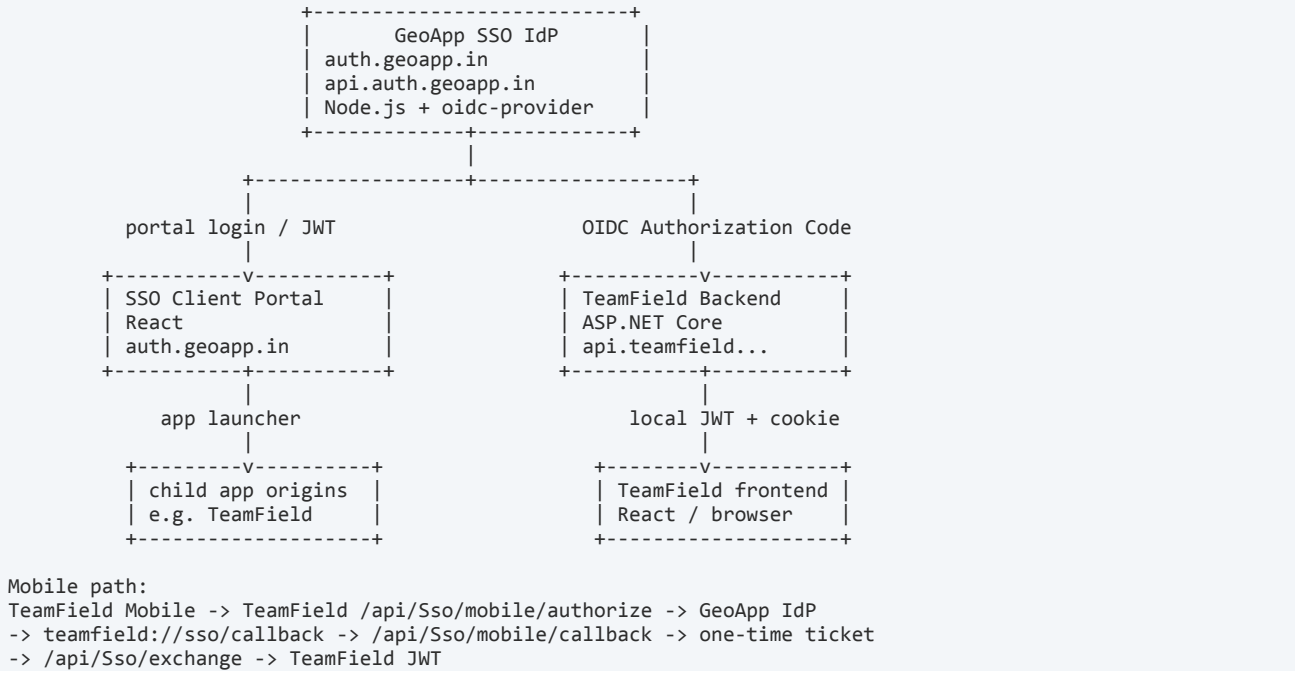
# 2. Repository & System Inventory

| Repository | Role | Primary stack | Key SSO responsibility |
|---|---|---|---|
| jay-geogroup/sso-backend | Central SSO / IdP | Node.js 22, TypeScript ESM, TypeORM, PostgreSQL 16, Redis, BullMQ, oidc-provider | Identity, OIDC endpoints, app registry, access control, portal JWTs, SSO session, provisioning orchestration |
| jay-geogroup/sso-frontend-client | GeoApp end-user portal | React, Redux Toolkit / RTK Query | Login, token refresh, app catalog, seamless child-app launch, OIDC interaction screens |
| jay-geogroup/sso-frontend-admin | SSO administration console | React/Vite, MUI, Redux Toolkit | Admin authentication and management UI for SSO users/apps/roles/access/provisioning |
| jay-geogroup/teamfield-backend | Child application backend / RP | ASP.NET Core | OIDC bootstrap, callback/state handling, ticket exchange, local JWT issuance, refresh cookie, logout, provisioning endpoint |
| jay-geogroup/teamfield-frontend | TeamField browser client | React/Vite | Consumes TeamField JWT, handles `/sso-callback`, stores only TeamField access token, refreshes silently |
| jay-geogroup/teamfield-mobileApp | TeamField mobile client | Ionic React, Capacitor Android | Native login UX, device ID, local TeamField session, mobile OIDC deep-link flow |

> **Branch snapshot**
> Current default branches observed: `main` for both SSO frontend repositories and `sso-backend`; `master` for TeamField backend/frontend/mobile. Treat branch choice as an implementation snapshot, not a protocol requirement.

# 3. Architecture Overview

## 3.1 Logical components




```
Mobile path:
TeamField Mobile -> TeamField /api/Sso/mobile/authorize -> GeoApp IdP
-> teamfield://sso/callback -> /api/Sso/mobile/callback -> one-time ticket
-> /api/Sso/exchange -> TeamField JWT
```

*Source: Derived from sso-backend SSO session/launcher code and TeamField SsoController contracts.*

## 3.2 Trust boundaries

| Boundary | Credential / artifact | Who may see/use it | Do not do |
|---|---|---|---|
| Browser <-> GeoApp IdP | OIDC session cookie | Browser and GeoApp auth host | Do not copy it into localStorage or application JSON APIs. |
| SSO backend <-> OIDC token endpoint | OIDC `client_id` + `client_secret` for confidential clients | Child-app backend only | Never embed the client secret in React or APK/IPA code. |
| SSO backend <-> child provisioning API | `X-Sso-Provisioning-Key` shared secret | SSO backend + child backend only | Never use the normal user JWT for provisioning. |
| Child backend <-> child frontend | Child-app JWT + optional refresh cookie | Browser/app + child API | Do not treat the child JWT as a GeoApp OIDC token. |

# 4. Authentication Models Used in GeoApp

## 4.1 Portal JWT session

The GeoApp portal has a conventional email/password login endpoint. The backend returns a JWT access token and refresh token. The React portal keeps credentials in its Redux state and retries failed API calls after a 401 using the refresh endpoint. This is the portal application session model.

```
const rawBaseQuery = fetchBaseQuery({
  baseUrl: API_BASE_URL,
  credentials: 'include',
  prepareHeaders: (headers, { getState }) => {
    const token = (getState() as RootState).auth.accessToken
```

```
    if (token) {
      headers.set('Authorization', `Bearer ${token}`)
    }
    return headers
  },
})
```

*Source: sso-frontend-client/src/store/api/authApi.ts*

> **Why this is not the child-app token**
> A successful portal login proves the user is authenticated to GeoApp. The OIDC session bridge makes that fact reusable by child applications; the child application still creates its own local application session.

## 4.2 OIDC session

The OIDC provider maintains its own session object and cookie. The portal login path explicitly synchronizes this OIDC session so that the next authorization request can proceed without asking for another password.

```
if (session.accountId !== accountId || !session.loginTs) {
  session.accountId = accountId;
  session.loginTs = Math.floor(Date.now() / 1000);
  session.amr = ["pwd"];
  session.touched = true;
}

await session.save(SESSION_TTL_SECONDS);

cookies.set(
  provider.cookieName("session"),
  session.id,
  longCookieOptions()
);
```

*Source: sso-backend/src/v1/services/ssoSession.service.ts*

## 4.3 TeamField local JWT session

TeamField does not keep the GeoApp password, nor does its browser need the GeoApp client secret. After the OIDC code is accepted, the TeamField backend issues a TeamField JWT. The frontend stores that child token under `jwt_access_token`.

```
const response = await axios.post('/api/Sso/exchange', {
  ticket,
  deviceId: getOrCreateDeviceId()
});
const token = response.data?.accessToken;
setSession(token);
```

*Source: teamfield-frontend/src/app/auth/services/jwt/JwtAuthProvider.tsx*

# 5. Core OIDC Provider Implementation (sso-backend)

## 5.1 OIDC endpoints and supported behavior

| Endpoint | Purpose | Notes |
|---|---|---|
| `/oidc/auth` | Authorization endpoint | Receives authorization requests from relying parties. |
| `/oidc/token` | Token endpoint | Exchanges authorization codes / refresh tokens. |
| `/oidc/introspection` | Token introspection | Enabled in provider configuration. |
| `/oidc/revocation` | Token revocation | Enabled in provider configuration. |
| `/oidc/end_session` | RP-initiated logout | Used by TeamField logout flow. |
| `/oidc/device` | Device authorization | Provider route is configured; child-app usage should be designed per client platform. |

## 5.2 Claims and identity mapping

The provider resolves a GeoApp user into an OIDC account whose subject is the user identity. The current provider configuration exposes `openid`, `profile`, `email`, and `offline_access`, with identity claims centered on subject, name, email, and `email_verified`.

## 5.3 OIDC TTLs observed

| Artifact | Current TTL | Operational meaning |
|---|---|---|
| Access token | 3600 sec | Child application access token validity. |
| Authorization code | 60 sec | Short-lived one-time code; exchange quickly. |
| ID token | 3600 sec | Identity assertion lifetime. |
| Refresh token | 14 days | OIDC refresh duration in provider config; TeamField additionally has its own local refresh mechanism. |
| SSO session | 30 days | Browser session used for silent SSO launches. |

## 5.4 Production signing keys

The OIDC provider is deliberately strict in production: it requires `OIDC_SIGNING_KEYS_PATH` pointing to a valid JWKS file. Missing or malformed signing keys fail startup. This is important because child applications validate OIDC-signed material against stable signing keys and should not encounter accidental development quick-start keys in production.

```
if (nodeEnv !== "production") {
  return undefined;
}

if (!filePath) {
  throw new JwksConfigurationError(
    "OIDC_SIGNING_KEYS_PATH is required in production"
  );
}

const jwks = await readJwksFile(filePath);
```

*Source: sso-backend/src/v1/oidc/jwks.ts*

## 5.5 Client registration is persisted and synchronized to oidc-provider

The application registry stores the client configuration in PostgreSQL. Registration then upserts the same client definition into oidc-provider. The code comments state that oidc-provider caches clients in memory, so production changes may require an application restart to guarantee the runtime provider is using the new configuration.

> **Operational rule**
> After creating or materially changing an OIDC application in the SSO admin, include an SSO backend restart in the rollout checklist unless the implementation is later changed to a live client store refresh.

# 6. Portal Login to Silent SSO Session

## 6.1 What happens when a user signs in

1. User submits email/password to the GeoApp portal login endpoint.
2. SSO backend validates the password and creates its normal application session / JWT credentials.
3. The same login request calls `createOrRefreshSsoSession(...)` and writes the signed oidc-provider session cookie on the auth host.
4. The portal loads the user's accessible applications from `/v1/auth/apps`.
5. When the user selects a first-party application, the portal opens the SSO launcher URL for that application.
6. The launcher resolves the first-party app's `/api/Sso/authorize?deviceId=...` bootstrap endpoint rather than sending the user directly to an application password form.
7. The child application creates its OIDC state/PKCE/nonce and redirects the browser to `/oidc/auth`.

8. The existing GeoApp OIDC session is recognized, so a second password prompt is not required.

```
if (origin && app.isFirstParty && app.redirectUris?.length) {
  const url = new URL(`${origin}/api/Sso/authorize`);
  url.searchParams.set("deviceId", accountId);
  return url.toString();
}

if (app.launchUrl) return app.launchUrl;
```

*Source: sso-backend/src/v1/services/ssoSession.service.ts*

### 6.2 Why the OIDC session is synchronized

Without the synchronization step, the portal JWT would authenticate the user to `api.auth.geoapp.in`, but the oidc-provider would not necessarily know that the browser session is already authenticated. The code explicitly solves this gap so first-party app launch feels like a single sign-on experience rather than a chain of independent logins.

## 7. Application Registration Model

### 7.1 Application record

The `applications` table is the single configuration record for a child application. It contains protocol settings and also launcher/access/provisioning metadata.

| Field | Meaning | Example / guidance |
|---|---|---|
| id | Internal application record UUID | Used by SSO admin/access APIs and portal app records; not a secret. |
| clientId | OIDC client identifier | Stable identifier used in the authorization and token flow. |
| clientSecret | OIDC confidential-client secret, encrypted at rest | Server-side only for confidential clients. |
| redirectUris | Allowed authorization callback URIs | Exact-match, HTTPS in production. |
| postLogoutRedirectUris | Allowed logout return URIs | Use exact registered URIs. |
| allowedGrantTypes | Permitted OIDC grants | Current defaults include authorization_code and refresh_token. |
| allowedScopes | Permitted scopes | Current defaults include openid/profile/email. |
| platformType | web or mobile | Use to describe target platform and inform integration behavior. |
| launchUrl | User-facing app launch URL | Portal uses this to start application launch. |
| isFirstParty | Whether app participates in seamless GeoApp launch | First-party app launch can use `/api/Sso/authorize` bootstrap behavior. |
| tokenEndpointAuthMethod | `client_secret_basic` or `none` | Confidential web backend usually uses client_secret_basic. |
| provisioningEnabled | Whether SSO pushes user-access events to app | Independent from OIDC login itself. |
| provisioningUrl / revocationUrl | Child endpoints for user lifecycle sync | HTTPS and SSRF-safe target validation. |
| provisioningSecret | Separate S2S secret, encrypted at rest | Never put in browser/mobile code. |

### 7.2 Registration code path

```
const { clientId, plainSecret, encryptedSecret } =
  await credentialService.generateClientCredentials();

const application = repository.create({
  ...params,
  clientId,
  clientSecret: encryptedSecret,
  allowedGrantTypes: params.allowedGrantTypes ??
    ["authorization_code", "refresh_token"],
  allowedScopes: params.allowedScopes ??
    ["openid", "profile", "email"],
});
```

*Source: sso-backend/src/v1/services/application.service.ts*

The credential service generates a GeoApp-prefixed client ID and a high-entropy secret, encrypts the secret for storage, and returns the plaintext secret only for the registration/rotation response.

```
const clientId = `geoapp_${uuidv4().replace(/-/g, "").substring(0, 24)}`;
const plainSecret = uuidv4();
const encryptedSecret = encryptSecret(plainSecret);
return { clientId, plainSecret, encryptedSecret };
```

*Source: sso-backend/src/v1/services/credential.service.ts*

# 8. Credentials: What You Receive, Where They Go, and Why

| Credential / field | Secret? | Where it belongs | Why it exists | Do not |
|---|---|---|---|---|
| Application ID (`id`) | No | SSO admin / access-control configuration | Identifies the application record for SSO management and user access assignment. | Do not confuse with `clientId`; it is not the OIDC client identifier. |
| `clientId` | No (identifier) | Child backend config; can be present in public protocol metadata | Tells GeoApp which OIDC client is requesting authorization/token exchange. | Do not assume knowing the client ID is sufficient to authenticate a confidential client. |
| `clientSecret` | YES | Child backend secret store only | Authenticates the confidential client at the OIDC token endpoint. | Never commit, log, expose to React, or bundle into an APK. |
| `redirectUris` | No | SSO application registration + child config | Prevents authorization responses from being redirected to an unapproved callback. | Do not use wildcards or broad origins. |
| `postLogoutRedirectUris` | No | SSO registration | Controls where RP-initiated logout may return the browser. | Do not use attacker-controlled URLs. |
| `provisioningSecret` | YES | Child backend secret store + SSO backend encrypted store | Authenticates server-to-server user lifecycle callbacks. | Never reuse the OIDC client secret, user JWT, or frontend token. |
| OIDC signing JWKS | YES for private parts | SSO IdP server filesystem/secret volume | Signs OIDC assertions/tokens and must remain under IdP control. | Do not commit private signing keys. |
| `PROVISIONING_ENCRYPTION_KEY` | YES | SSO backend environment / secret store | Encrypts provisioning secrets at rest using AES-256-GCM key derivation. | Do not store it in source code. |

**One-time secret display**

The application registration and rotation APIs return plaintext client/provisioning secrets once. Treat the response like a password reset code: copy it directly into the backend secret store, verify connectivity, and do not paste it into tickets, Slack, logs, browser local storage, or source control.

## 8.1 What the client secret actually does

A confidential OIDC client is an application capable of protecting credentials outside the browser. In this implementation, the TeamField backend uses the client secret during the authorization-code exchange. The browser only handles redirects and the one-time ticket that TeamField issues after it has accepted the OIDC response.

## 8.2 Why the provisioning secret is separate

Provisioning answers a different trust question: “Is this request genuinely coming from the GeoApp SSO service?” It is not an end-user login credential. TeamField enforces this separation by implementing a dedicated `SsoProvisioning` authentication scheme and requiring the `X-Sso-Provisioning-Key` header.

# 9. Web Application Integration - TeamField

## 9.1 Current TeamField browser flow

9. User opens TeamField or clicks TeamField from the GeoApp application launcher.
10. TeamField `/api/Sso/authorize` builds the authorization request and redirects to the GeoApp OIDC authorization endpoint.
11. GeoApp sees the browser's already-authenticated SSO session and completes the authorization step without another password prompt when policy allows.
12. GeoApp redirects to `https://teamfield.geoapp.in/api/Sso/callback` with `code` and `state`.
13. TeamField validates/exchanges the code and state server-side and responds with a one-time JSON `ticket`.
14. TeamField frontend `/sso-callback` route calls `/api/Sso/exchange` with `{ ticket, deviceId }`.
15. TeamField backend issues a TeamField JWT and, when a device ID is supplied, an HttpOnly Secure SameSite=None refresh-token cookie.
16. TeamField frontend stores the TeamField JWT and uses it for normal API authorization.

```
[AllowAnonymous]
[HttpGet("authorize")]
public async Task<IActionResult> Authorize()
{
    var url = await _ssoService.BuildAuthorizeUrl(
        Request.Query["deviceId"].FirstOrDefault());
    return Redirect(url);
}
```

*Source: teamfield-backend/TeamField.API/Controllers/SsoController.cs*

```
[AllowAnonymous]
[HttpGet("callback")]
public async Task<IActionResult> Callback(
    string? code, string? state, string? error)
{
    if (!string.IsNullOrWhiteSpace(error) ||
        string.IsNullOrWhiteSpace(code) ||
        string.IsNullOrWhiteSpace(state))
    {
        await _ssoService.ConsumeAuthorizationStateAsync(state ?? string.Empty);
        return BadRequest(new { error = "authorization_failed" });
    }

    var ticket = await _ssoService.HandleCallback(code, state);
    return Ok(new { ticket });
}
```

*Source: teamfield-backend/TeamField.API/Controllers/SsoController.cs*

```
const response = await axios.post('/api/Sso/exchange', {
  ticket,
  deviceId: getOrCreateDeviceId()
});
const token = response.data?.accessToken;
setSession(token);
```

*Source: teamfield-frontend/src/app/auth/services/jwt/JwtAuthProvider.tsx*

## 9.2 TeamField OIDC configuration template

```
"Sso": {
  "Issuer": "https://auth.geoapp.in",
  "ClientId": "<CLIENT_ID_FROM_SSO_REGISTRATION>",
  "ClientSecret": "<CLIENT_SECRET_FROM_SSO_REGISTRATION>",
  "RedirectUri": "https://teamfield.geoapp.in/api/Sso/callback",
  "MobileRedirectUri": "teamfield://sso/callback",
  "Scopes": "openid profile email",
  "FrontendUrl": "https://teamfield.geoapp.in",
  "ProvisioningEnabled": false,
  "ProvisioningAuthSecret": "<PROVISIONING_SECRET_IF_ENABLED>"
}
```

*Source: TeamField.Domain.Models.Sso.SsoOptions + current production-oriented configuration shape*

### 9.3 Callback URI discrepancy to verify

The TeamField backend explicitly registers/uses the backend callback `https://teamfield.geoapp.in/api/Sso/callback`. The TeamField frontend `.env.example` comments separately describe `/sso-callback` as the frontend callback route. The backend controller itself explains that the browser callback route is frontend `/sso-callback` but the OIDC callback endpoint is the API endpoint. This is a valid two-step bridge only if the current frontend/router and backend deployment wiring match. Before changing any URL, verify the deployed navigation path and reverse proxy configuration in the target environment.

> **Do not blindly change the registered redirect URI**
> OIDC redirect URIs are exact security boundaries. The value registered in SSO must equal the URI actually used by the TeamField backend in the authorization-code request. A frontend route can be the user-facing callback page while the server endpoint remains the OIDC redirect target.

## 10. TeamField Frontend Session Handling

```
export function getAccessToken(): string | null {
  return localStorage.getItem(jwtAuthConfig.tokenStorageKey);
}

export function setAccessToken(token: string): void {
  localStorage.setItem(jwtAuthConfig.tokenStorageKey, token);
  localStorage.setItem(jwtAuthConfig.ssoSessionKey, 'true');
}
```

*Source: teamfield-frontend/src/app/auth/services/jwt/ssoSession.ts*

The TeamField browser client intentionally stores only the TeamField JWT. Its own session provider can refresh silently using `/api/Auth/refresh` and the persisted refresh-token cookie. This isolates the GeoApp OIDC credential boundary from the child application UI.

```
if (window.location.pathname.startsWith('/sso-callback')) {
  setIsLoading(false);
  setAuthStatus('unauthenticated');
  return;
}

const token = getAccessToken();
...
if (!isTokenValid(usableToken)) {
  usableToken = await refreshToken();
}
```

*Source: teamfield-frontend/src/app/auth/services/jwt/JwtAuthProvider.tsx*

> **Security implication**
> The child frontend does not need to parse or persist a GeoApp ID token, client secret, or provisioning secret. Its only responsibility is to consume the TeamField session produced by the backend.

## 11. Mobile Application Integration - Ionic React

### 11.1 Current mobile contract

The TeamField backend exposes dedicated JSON endpoints for mobile: `/api/Sso/mobile/authorize`, `/api/Sso/mobile/callback`, and `/api/Sso/mobile/logout`. The mobile redirect URI is a custom scheme (`teamfield://sso/callback`). This lets the app open the authorization URL externally/in a browser context and then regain control through a deep link.

```
[AllowAnonymous]
[HttpGet("mobile/authorize")]
public async Task<IActionResult> MobileAuthorize()
{
    var url = await _ssoService.BuildMobileAuthorizeUrl(
        Request.Query["deviceId"].FirstOrDefault());
```

```
    return Ok(new { url });
}

[AllowAnonymous]
[HttpPost("mobile/callback")]
public async Task<IActionResult> MobileCallback(
    [FromBody] MobileCallbackRequest request)
{
    var ticket = await _ssoService.HandleCallback(request.Code, request.State);
    return Ok(new { ticket });
}
```

*Source: teamfield-backend/TeamField.API/Controllers/SsoController.cs*

### 11.2 Android/deep-link requirement

- Register the custom URI scheme `teamfield://` in the Android application so the OS can route `teamfield://sso/callback?...` back to the app.
- Do not ship the OIDC client secret inside the Ionic bundle or Android APK. The TeamField backend remains the confidential OIDC client in the current architecture.
- Persist the stable device ID in the application's local storage. The supplied mobile repository already exposes a `getOrCreateDeviceId()` pattern through its authentication API integration.
- After deep-link handling, POST only the returned `code` and `state` to `/api/Sso/mobile/callback`; then send the one-time ticket to `/api/Sso/exchange` with the device ID.
- Store the resulting TeamField access/refresh material according to the mobile app's secure-storage policy. If the current app remains localStorage-oriented, harden this for production with platform secure storage / keychain-backed storage rather than treating WebView localStorage as a high-security vault.

```
export const authApi = api.injectEndpoints({
  endpoints: (builder) => ({
    refresh: builder.mutation<{ accessToken: string }, { deviceId: string }>({
      query: () => ({
        url: `/Auth/refresh`,
        method: 'POST',
        params: { deviceId: getOrCreateDeviceId() }
      })
    }),
    logout: builder.mutation<{ success: boolean }, void>({
      query: () => ({
        url: '/Auth/logout',
        method: 'GET',
        params: { deviceId: getOrCreateDeviceId() }
      })
    })
  })
});
```

*Source: teamfield-mobileApp/src/app/Auth/authApi.ts*

> **Verification status**
> The mobile repository was inspected at package/auth entry-point level and the backend mobile contract was verified. A complete source-to-source mapping of the mobile SSO UI/deep-link event handler was not independently retrievable through the repository connector, so this guide treats the backend mobile endpoint contract as authoritative and marks the native callback handler as an integration verification step.

## 12. User Access, Roles, and Application Launcher

### 12.1 Application access is not the same as registration

Registering an OIDC client only creates the application identity. A user still needs an active `UserApplicationAccess` row for the application to appear as accessible in the portal. The access row points to an application-scoped role and records grant/hold metadata.

```
if (access.status === AccessStatus.ACTIVE) {
  accessible.push({ ...app, accessStatus: "ACTIVE" });
} else {
  requested.push({ ...app, accessStatus: access.status });
}
```

*Source: sso-backend/src/v1/controllers/auth.controller.ts*

### 12.2 Portal launcher

```
const launcherBase = import.meta.env.VITE_OIDC_INTERACTION_BASE_URL
  const url = `${launcherBase}/v1/oidc/apps/${app.appId}/launch`
  window.open(url, '_blank')
```

*Source: sso-frontend-client/src/features/dashboard/AppDashboard.tsx*

The portal therefore has three distinct concepts: the application registry, the user's application access grant, and the launch URL. A child app can be perfectly registered but still unavailable to a user until access is granted.

### 12.3 Roles

The SSO backend has a generic `ApplicationRole` model and a `roleId` reference from the access row. During provisioning, the selected role code can be forwarded to the child application. This creates a useful pattern: GeoApp controls which application the user may access, while the child application remains responsible for enforcing its own permission model using the provisioned role mapping.

## 13. Automatic User Provisioning

### 13.1 Why provisioning is separate from login

OIDC proves user identity; provisioning makes that identity usable inside a child application's local user database. The current implementation deliberately separates those concerns. If a user is granted TeamField access, SSO can asynchronously tell TeamField to create/activate its local user. If access is revoked, SSO can send a deprovision event.

### 13.2 Data flow

17. Admin grants application access using the SSO access API.
18. After the access row is committed, the SSO backend records a provisioning-tracking row.
19. The event is enqueued into BullMQ queue `sso-provisioning`.
20. The provisioning worker sends the lifecycle event to the registered child endpoint using the shared provisioning secret.
21. The child application authenticates the header `X-Sso-Provisioning-Key` with its dedicated `SsoProvisioning` scheme.
22. Success moves the provisioning record to COMPLETED; failures are retained with a safe error string and retry metadata.

```
const idempotencyKey =
  `${params.event}:${params.applicationId}:${params.userId}:${params.applicationAccessId}`;

record = provisioningRepo.create({
  ...params,
  roleCode: params.roleCode ?? null,
  idempotencyKey,
  status: ProvisioningStatus.PENDING,
});
await provisioningRepo.save(record);
await enqueueProvisioningJob(...);
```

*Source: sso-backend/src/v1/services/provisioning.service.ts*

```
export const provisioningQueue = new Queue(
  "sso-provisioning",
  {
    connection,
    prefix: config.redis_prefix,
    defaultJobOptions: {
      attempts: config.provisioning_max_attempts,
      backoff: {
        type: "exponential",
        delay: 3000,
      }
    }
  }
);
```

*Source: sso-backend/src/v1/queues/provisioning.queue.ts*

### 13.3 TeamField provisioning endpoint security

```
public const string SchemeName = "SsoProvisioning";
public const string HeaderName = "X-Sso-Provisioning-Key";

if (!FixedTimeEquals(provided.ToString(), configured))
    return Task.FromResult(
      AuthenticateResult.Fail("Invalid SSO provisioning credential"));

var claims = new[]
{
    new Claim(ClaimTypes.Name, "geoapp-sso-provisioning"),
    new Claim(ClaimTypes.Role, "SsoProvisioningService")
};
```

*Source: teamfield-backend/TeamField.API/Authentication/SsoProvisioningAuthenticationHandler.cs*

This dedicated authentication scheme is intentionally stronger than simply reusing a TeamField admin JWT. A normal user token must not be able to invoke an internal lifecycle endpoint that can create or deactivate application users.

### 13.4 Provisioning secret lifecycle

```
const plainSecret = generateProvisioningSecret();
app.provisioningSecret = encryptSecret(plainSecret);
await appRepo.save(app);
return plainSecret;
```

*Source: sso-backend/src/v1/helpers/secretCrypto.ts + credential.service.ts*

The provisioning secret is encrypted at rest using AES-256-GCM. The platform derives the encryption key from `PROVISIONING_ENCRYPTION_KEY`; if that value is absent, provisioning secret creation/rotation fails.

## 14. Logout and Session Revocation

### 14.1 TeamField browser logout

23. TeamField reads the child refresh-token cookie and revokes the token server-side.
24. The refresh-token cookie is deleted from the browser.
25. TeamField asks its SSO service to build a GeoApp end-session URL.
26. The browser is redirected to the GeoApp OIDC end-session endpoint to terminate the shared SSO session.

```
var cookieToken = Request.Cookies[RefreshTokenCookieName];
if (!string.IsNullOrWhiteSpace(cookieToken))
    await _authService.RevokeRefreshTokenAsync(cookieToken);

Response.Cookies.Delete(RefreshTokenCookieName, new CookieOptions {
    HttpOnly = true,
    Secure = true,
    SameSite = SameSiteMode.None
});

var url = await _ssoService.BuildLogoutUrl(null);
return Redirect(url.ToString());
```

*Source: teamfield-backend/TeamField.API/Controllers/SsoController.cs*

### 14.2 Portal logout

The SSO portal logout path revokes sessions and refresh tokens, blacklists the presented access token in Redis for its remaining one-hour class of lifetime, and records an audit log. Password reset/change operations also revoke all user sessions, which is the correct behavior for credential rotation.

```
await sessionService.revokeAllUserSessions(user.id);
await sessionService.revokeAllUserRefreshTokens(user.id);

if (token) {
  await redis.set(`blackListed-${token}`, true, 3600);
}
```

*Source: sso-backend/src/v1/controllers/portal.controller.ts*

# 15. Configuration & Environment Variables

## 15.1 SSO backend

```
NODE_ENV=production
APP_URL=https://auth.geoapp.in

DB_HOST=<postgres-host>
DB_PORT=5432
DB_USERNAME=<db-user>
DB_PASSWORD=<db-password>
DB_DATABASE=geoapp_sso

REDIS_HOST=<redis-host>
REDIS_PORT=6379

JWT_ENCRYPTION=<long-random-secret>
JWT_EXPIRATION=1h
JWT_REFRESH_EXPIRATION=7

OIDC_ISSUER=https://auth.geoapp.in
OIDC_SIGNING_KEYS_PATH=/var/www/geoapp-sso-secrets/oidc-jwks.json
OIDC_COOKIE_DOMAIN=auth.geoapp.in

PROVISIONING_ENCRYPTION_KEY=<long-random-secret>
PROVISIONING_HTTP_TIMEOUT_MS=10000
PROVISIONING_MAX_ATTEMPTS=5
SSRF_ALLOW_PRIVATE_TARGETS=false
```

*Source: sso-backend/.env.example + config.ts*

## 15.2 Portal frontend

```
VITE_API_BASE_URL=https://api.auth.geoapp.in
VITE_OIDC_INTERACTION_BASE_URL=https://auth.geoapp.in
```

*Source: sso-frontend-client/.env.example*

## 15.3 TeamField backend

```
Sso__Issuer=https://auth.geoapp.in
Sso__ClientId=<CLIENT_ID>
Sso__ClientSecret=<CLIENT_SECRET>
Sso__RedirectUri=https://teamfield.geoapp.in/api/Sso/callback
Sso__MobileRedirectUri=teamfield://sso/callback
Sso__Scopes=openid profile email
Sso__FrontendUrl=https://teamfield.geoapp.in
Sso__ProvisioningEnabled=false
Sso__ProvisioningAuthSecret=<PROVISIONING_SECRET_IF_ENABLED>
```

*Source: TeamField SsoOptions model and current appsettings structure; use environment/secret-store values in production*

**Secret-store recommendation**
Use environment injection, Azure Key Vault, AWS Secrets Manager, Kubernetes Secrets, HashiCorp Vault, or your platform's equivalent. The important property is that deploy-time secret injection replaces committed secret material.

# 16. New Application Onboarding Runbook

## 16.1 Decide the client architecture first

| Decision | Recommended choice for current GeoApp pattern | Reason |
|---|---|---|
| Server-backed web application | Confidential OIDC client + client_secret_basic | Backend can securely hold the client secret and perform the code exchange. |
| SPA that talks to IdP directly | Public client + PKCE + no secret | Browser cannot safely protect a long-lived client secret. |
| Mobile app with backend mediation | Backend remains confidential client; mobile gets deep-link + ticket | Keeps secret out of APK and centralizes code/state handling. |

| | | |
|---|---|---|
| Mobile app direct to IdP | Public OIDC client + PKCE | Only if the mobile app is intentionally the OIDC relying party and the platform supports secure redirect handling. |

## 16.2 Registration checklist

27. Create the application in SSO Admin with a stable name and description.
28. Choose `platformType` (`web` or `mobile`).
29. Set `launchUrl` to the browser/app entry point the user should reach from the portal.
30. Set `isFirstParty=true` when the child app is part of GeoApp and will implement the first-party bootstrap convention.
31. Register exact `redirectUris` for every environment. Keep localhost entries separate from production.
32. Register exact `postLogoutRedirectUris` when RP-initiated logout is supported.
33. Choose `allowedScopes`, normally `openid profile email` unless a specific future scope is intentionally added.
34. Choose `allowedGrantTypes`, normally `authorization_code` and `refresh_token` for the current provider model.
35. Choose token endpoint authentication. For a confidential backend, use `client_secret_basic`.
36. Enable provisioning only if the child app has the internal lifecycle endpoints ready and can store the provisioning secret server-side.
37. Copy the one-time `clientSecret` into the child backend secret store. Never place it in frontend `.env` files that start with `VITE_`.
38. If provisioning is enabled, copy the one-time provisioning secret into the child backend secret store as `Sso:ProvisioningAuthSecret` (or equivalent secret-store mapping).
39. Grant the appropriate application role/access to test users.
40. Restart/redeploy the SSO backend if required for its OIDC client cache to pick up registration changes.
41. Run browser, refresh, logout, denied-access, and provisioning tests before announcing the integration complete.

## 16.3 Example registration payload - confidential web child app

```
{
  "name": "Example Child App",
  "description": "Example GeoApp first-party application",
  "redirectUris": [
    "https://example.geoapp.in/api/Sso/callback"
  ],
  "postLogoutRedirectUris": [
    "https://example.geoapp.in/"
  ],
  "allowedGrantTypes": [
    "authorization_code",
    "refresh_token"
  ],
  "allowedScopes": [
    "openid",
    "profile",
    "email"
  ],
  "platformType": "web",
  "logoUrl": "https://example.geoapp.in/logo.png",
  "launchUrl": "https://example.geoapp.in",
  "isFirstParty": true,
  "tokenEndpointAuthMethod": "client_secret_basic",
  "provisioning": {
    "enabled": false
  }
}
```

*Source: Shape derived from POST /v1/applications controller input fields; validate against current admin API contract before automation.*

## 16.4 What to do immediately after registration

| Step | Artifact | Owner | Done when |
|---|---|---|---|
| 1 | clientId | Child backend developer | Configured in child backend, visible to protocol code. |
| 2 | clientSecret | Child backend / DevOps | Stored in secret manager and never committed. |
| 3 | redirectUri | SSO admin + child backend | Exact same URL is used in the authorization request. |

| | | | |
|---|---|---|---|
| 4 | OIDC issuer | Child backend | `https://auth.geoapp.in` is used as provider issuer. |
| 5 | provisioning secret (optional) | SSO + child backend | Header authentication works against the internal provisioning endpoint. |
| 6 | user access + role | SSO admin | Test user sees the app in the portal launcher. |
| 7 | runtime reload | SSO DevOps | Provider recognizes the new client config. |

# 17. End-to-End Request Flows

## 17.1 Web SSO sequence

```
Browser
  |
  | POST /v1/auth/login (email/password)
  v
SSO API
  |-- issue portal access/refresh tokens
  |-- create/refresh oidc-provider session
  |-- set signed SSO session cookie
  |
  | GET /v1/auth/apps
  v
Portal
  |
  | GET /v1/oidc/apps/{appId}/launch
  v
SSO Launcher
  |
  | 302 -> TeamField /api/Sso/authorize?deviceId=...
  v
TeamField API
  |
  | 302 -> https://auth.geoapp.in/oidc/auth?...state...code_challenge...
  v
GeoApp OIDC
  |
  | Existing SSO session recognized
  | 302 -> TeamField /api/Sso/callback?code=...&state=...
  v
TeamField API
  |
  | validate state + exchange code
  | return { ticket }
  v
TeamField /sso-callback UI
  |
  | POST /api/Sso/exchange { ticket, deviceId }
  v
TeamField API
  |
  | issue TeamField JWT + refresh cookie
  v
TeamField App
```

## 17.2 Mobile SSO sequence

```
Mobile App
  |
  | GET /api/Sso/mobile/authorize?deviceId=...
  v
TeamField API
  |
  | returns { url }
  v
Browser / Custom Tab
  |
  | -> GeoApp OIDC /oidc/auth
  | <- redirect to teamfield://sso/callback?code=...&state=...
```

```
  v
Mobile App
  |
  | POST /api/Sso/mobile/callback { code, state }
  v
TeamField API
  |
  | returns { ticket }
  v
Mobile App
  |
  | POST /api/Sso/exchange { ticket, deviceId }
  v
TeamField API
  |
  | TeamField JWT (+ refresh mechanism)
  v
Mobile Session
```

### 17.3 Provisioning sequence

```
SSO Admin
  |
  | grant user -> application access
  v
SSO API / DB
  |
  | record access
  | create provisioning row
  | enqueue BullMQ job
  v
sso-provisioning queue
  |
  | POST https://api.teamfield.geoapp.in/internal/sso/users/provision
  | X-Sso-Provisioning-Key: <secret>
  v
TeamField API
  |
  | SsoProvisioning auth scheme
  | create/activate local user
  v
TeamField DB

On revoke: same pattern with /deprovision
```

## 18. Troubleshooting Guide

| Symptom | Likely cause | What to check | Fix |
|---|---|---|---|
| `invalid_client` | Wrong client ID/secret or token auth mismatch | SSO registration, child backend secret store, token endpoint auth method | Rotate secret if needed; update backend only; restart child/SSO runtime as required. |
| `invalid_grant` / code failure | Authorization code expired, reused, or redirect mismatch | 60-second code TTL, exact redirect URI, code reuse | Start a new login flow; verify redirect URI exactness. |
| Second login prompt appears | OIDC session not created or cookie not available | Portal login path, auth host cookie, browser cookie policy | Verify `createOrRefreshSsoSession`, cookie domain, HTTPS, and same-site behavior. |
| TeamField `/sso-callback` gets blocked / blank | Frontend auth guard intercepts callback route | `Authentication.tsx` and router | Ensure callback route bypasses normal “must already be authenticated” gate. |
| Ticket exchange returns `invalid_ticket` | Ticket expired/consumed or wrong endpoint | One-time ticket lifecycle, code/state pairing | Perform new OIDC flow; never retry a consumed ticket. |
| Provisioning returns 401 | Missing/wrong provisioning header | `X-Sso-Provisioning-Key`, child `Sso:ProvisioningAuthSecret` | Rotate and synchronize the provisioning secret. |
| Provisioning stuck PENDING/FAILED | Worker/Redis or downstream endpoint failure | BullMQ queue, Redis, provisioning status, retry count | Inspect queue and retry from SSO admin after verifying child availability. |
| New app not recognized after registration | oidc-provider in-memory client cache | SSO process startup time / deployment | Restart the SSO backend after registration changes. |

| | | | |
|---|---|---|---|
| Refresh loops / logout after idle | Refresh token/cookie mismatch | Child `/Auth/refresh`, refresh cookie, device ID | Verify cookie attributes, device ID stability, and server-side token store. |
| Mobile login returns to browser but not app | Deep-link registration issue | Android intent filter / custom scheme | Verify `teamfield://sso/callback` is registered and callback parser handles `code` + `state`. |

### 18.1 Debug logging without leaking secrets

- Log request correlation IDs, client/application IDs, route names, status codes, and timing.
- Never log `clientSecret`, provisioning secret, access token, refresh token, authorization code, or one-time ticket values.
- When troubleshooting callback failures, log whether `code` and `state` are present, not their literal values.
- When troubleshooting provisioning, log application ID, user ID, event type, attempt number, and sanitized error reason.

## 19. Security Review & Mandatory Remediation

> **CRITICAL: committed secrets found**
> The supplied TeamField `TeamField.API/appsettings.json` contains literal values for OIDC client secret, SMTP credentials, AWS access key/secret key, and the application JWT signing secret. Do not copy these values into any new configuration. Treat them as compromised and rotate them immediately.

### 19.1 Required actions

42. Rotate the TeamField OIDC client secret in SSO Admin. Update only the TeamField backend secret store, then restart TeamField backend.
43. Rotate the TeamField application JWT signing secret. This invalidates existing TeamField JWTs and should be rolled out deliberately.
44. Rotate the SMTP credential and replace the committed value with a secret-store reference.
45. Rotate the AWS access key pair (or, preferably, replace static keys with IAM role / workload identity).
46. Remove committed secrets from the repository history where organizational policy requires full removal, not merely a new commit that deletes the file.
47. Verify no secrets exist in frontend `VITE_*` values or mobile build-time variables.
48. Add secret scanning / pre-commit / CI controls to block reintroduction.

### 19.2 Current good security patterns worth preserving

- OIDC client secrets are encrypted at rest rather than hashed because the provider must recover the plaintext to validate token-endpoint authentication.
- Provisioning secrets use AES-256-GCM at rest and are revealed once.
- Provisioning requests use a dedicated authentication scheme and constant-time secret comparison.
- Production OIDC signing keys are mandatory and startup fails when missing or invalid.
- Provisioning has deterministic idempotency keys and background retry handling.
- Password change/reset operations revoke active sessions.

### 19.3 Additional hardening recommendations

- Prefer authorization-code + PKCE for every browser/mobile relying-party flow, including confidential backends when your library supports it cleanly.
- Keep redirect URIs exact and environment-specific.
- Use HTTPS everywhere in production and make secure-cookie behavior environment-aware only for local development.
- Review the TeamField backend CORS list and keep only required trusted origins.
- Use platform secure storage for mobile refresh/access material where feasible.
- Define a formal client-secret rotation playbook with overlap / maintenance windows if future provider support permits dual secrets.

# 20. Recommended Standardization Plan

## Phase 1 - Make the existing pattern the official platform contract

- Officially standardize `/{api}/Sso/authorize`, `/callback`, `/exchange`, `/logout` for browser child apps.
- Officially standardize `/api/Sso/mobile/authorize`, `/mobile/callback`, `/mobile/logout` for mediated mobile clients.
- Publish the exact application-registration schema and one-time credential handling policy.
- Publish the provisioning contract: `USER_ACCESS_GRANTED`, `USER_ACCESS_REVOKED`, `USER_ACCESS_UPDATED`, `USER_IDENTITY_UPDATED` plus idempotency expectations.

## Phase 2 - Extract a reusable child-app SDK / package

Future applications should not hand-roll state, nonce, PKCE, ticket handling, logout, and refresh logic repeatedly. The preferred direction is a small GeoApp integration package or documented reference implementation for Node.js, ASP.NET Core, React SPA, and Ionic/Capacitor.

## Phase 3 - Improve operational visibility

- Add application onboarding status: Registered -> Configured -> Health Checked -> User Access Granted -> Provisioning Healthy.
- Add a per-application “Test OIDC” button in admin that runs discovery, authorization bootstrap validation, and provisioning health checks where safe.
- Expose provisioning attempt counts and latest safe failure reason in the admin UI (already partially supported by backend tracking).
- Introduce client configuration versioning so it is obvious when a runtime still has an older in-memory configuration.

## Phase 4 - Stronger cryptographic and secret-management posture

- Move OIDC signing keys to a managed key system or tightly controlled secret volume with documented rotation.
- Use workload identity / managed identity for AWS or Azure resources where possible.
- Centralize audit events for client-secret rotation, provisioning-secret rotation, app changes, access grants/revocations, and logout.

# 21. Developer Checklist

| Area | Checklist item | Required |
|---|---|---|
| Registration | Application has exact production redirect URI(s). | Yes |
| Registration | Application has exact post-logout redirect URI(s). | When logout redirect is used |
| Registration | `authorization_code` and `refresh_token` grants are intentionally enabled. | Usually yes |
| Registration | Scopes are minimal (`openid profile email` unless more are truly needed). | Yes |
| Secrets | Client secret stored only on confidential backend. | Yes |
| Secrets | Provisioning secret stored only on backend(s). | When provisioning enabled |
| Web | Authorization request includes state and PKCE/nonce as supported by child integration. | Yes |
| Web | Callback route is reachable and not blocked by auth guard. | Yes |
| Web | Ticket is treated as one-time / short-lived. | Yes |
| Mobile | Custom deep link is registered. | For mobile |
| Mobile | No OIDC client secret is bundled in APK. | Yes |
| Access | Test user has active application access and correct application role. | Yes |
| Provisioning | Child internal endpoint returns 2xx with correct service header. | When enabled |

| | | |
|---|---|---|
| Provisioning | Retry/idempotency behavior has been tested. | When enabled |
| Runtime | SSO backend restarted after client registration if required by provider cache behavior. | Yes |
| Security | Committed secrets removed and rotated. | Mandatory |

## 22. Source Reference Map

| Repo | Important file / area | What this guide uses it for |
|---|---|---|
| sso-backend | src/v1/controllers/auth.controller.ts | Portal login, refresh, logout, accessible app list. |
| sso-backend | src/v1/controllers/application.controller.ts | Application registration/rotation/provisioning inputs and one-time secret responses. |
| sso-backend | src/v1/services/application.service.ts | Default app settings, credential generation, provider client sync. |
| sso-backend | src/v1/services/credential.service.ts | Client secret generation/rotation and provisioning secret generation. |
| sso-backend | src/v1/services/ssoSession.service.ts | OIDC session bridge, first-party pre-grant, launcher resolution. |
| sso-backend | src/v1/oidc/provider.ts | OIDC provider routes, scopes, claims, TTLs, interaction behavior, client sync. |
| sso-backend | src/v1/oidc/jwks.ts | Production signing key requirements. |
| sso-backend | src/v1/entities/Application.entity.ts | Application registration schema. |
| sso-backend | src/v1/entities/UserApplicationAccess.entity.ts | Per-user application access model. |
| sso-backend | src/v1/entities/ApplicationUserProvisioning.entity.ts | Provisioning tracking and idempotency. |
| sso-backend | src/v1/services/provisioning.service.ts | Queueing, retry, provisioning state changes. |
| sso-backend | src/v1/queues/provisioning.queue.ts | BullMQ queue and dedupe job ID. |
| sso-backend | src/v1/helpers/secretCrypto.ts | AES-256-GCM storage of secrets. |
| sso-backend | src/v1/services/session.service.ts | Session and refresh-token persistence/revocation. |
| sso-frontend-client | src/store/api/authApi.ts | Portal API calls, bearer auth, refresh-on-401. |
| sso-frontend-client | src/features/interaction/interactionClient.ts | OIDC interaction login/consent client. |
| sso-frontend-client | src/features/dashboard/AppDashboard.tsx | Application launcher behavior. |
| sso-frontend-admin | src/app/auth/Authentication.tsx | Admin frontend JWT auth gate. |
| teamfield-backend | TeamField.API/Controllers/SsoController.cs | Web/mobile OIDC bootstrap, callback, exchange, logout. |
| teamfield-backend | TeamField.API/Authentication/SsoProvisioningAuthenticationHandler.cs | Dedicated provisioning authentication. |
| teamfield-backend | TeamField.Domain/Models/Sso/SsoOptions.cs | Backend SSO configuration contract. |
| teamfield-frontend | src/app/auth/services/jwt/JwtAuthProvider.tsx | TeamField JWT session and SSO ticket exchange. |
| teamfield-frontend | src/app/auth/services/jwt/ssoSession.ts | Child token storage boundary. |
| teamfield-frontend | src/app/auth/Authentication.tsx | Special handling for `/sso-callback` route. |
| teamfield-mobileApp | src/app/Auth/authApi.ts | Device-aware local auth refresh/logout contract. |
| teamfield-mobileApp | src/App.tsx | Mobile app routing/auth entry point. |

*Repository source links in this guide refer to the exact GitHub repository paths supplied for the review. This document intentionally omits literal credentials found in committed configuration.*

# 23. Appendix: Example Registration Payloads

## 23.1 First-party web app without provisioning

```
{
  "name": "GeoReports",
  "description": "GeoApp internal reporting application",
  "redirectUris": [
    "https://georeports.geoapp.in/api/Sso/callback"
  ],
  "postLogoutRedirectUris": [
    "https://georeports.geoapp.in/"
  ],
  "allowedGrantTypes": ["authorization_code", "refresh_token"],
  "allowedScopes": ["openid", "profile", "email"],
  "platformType": "web",
  "logoUrl": "https://georeports.geoapp.in/logo.png",
  "launchUrl": "https://georeports.geoapp.in",
  "isFirstParty": true,
  "tokenEndpointAuthMethod": "client_secret_basic",
  "provisioning": {
    "enabled": false
  }
}
```

## 23.2 First-party web app with provisioning

```
{
  "name": "GeoReports",
  "redirectUris": [
    "https://georeports.geoapp.in/api/Sso/callback"
  ],
  "allowedGrantTypes": ["authorization_code", "refresh_token"],
  "allowedScopes": ["openid", "profile", "email"],
  "platformType": "web",
  "launchUrl": "https://georeports.geoapp.in",
  "isFirstParty": true,
  "tokenEndpointAuthMethod": "client_secret_basic",
  "provisioning": {
    "enabled": true,
    "url": "https://api.georeports.geoapp.in/internal/sso/users/provision",
    "revocationUrl": "https://api.georeports.geoapp.in/internal/sso/users/deprovision",
    "authType": "shared_secret"
  }
}
```

> **Provisioning endpoint design**
> The current SSO application model supports a primary provisioning URL and an optional dedicated revocation URL. For TeamField, those map naturally to `/internal/sso/users/provision` and `/internal/sso/users/deprovision`.

## 23.3 Mobile public-client alternative (only for direct OIDC ownership)

The current TeamField mobile architecture is backend-mediated, so it should continue using the TeamField backend as the confidential OIDC client. A public OIDC client with `token_endpoint_auth_method = none` is only appropriate when the mobile app itself is the OIDC client and uses PKCE directly. Do not change TeamField to this model without reworking its code/state/ticket architecture.

```
{
  "platformType": "mobile",
  "allowedGrantTypes": ["authorization_code", "refresh_token"],
  "allowedScopes": ["openid", "profile", "email"],
  "tokenEndpointAuthMethod": "none"
}
```

# Document Maintenance Notes

This guide is a code-derived snapshot based on the repositories available during review. The protocol concepts and integration contracts should remain stable, but file paths, endpoint behavior, default TTLs, and configuration names can

change. Whenever the SSO backend changes, update this document alongside the application registration contract and regression test suite.

**Recommended source-of-truth practice**

Keep the SSO protocol contract (registration fields, credentials, endpoints, required callbacks, provisioning payloads, and rotation procedures) in version-controlled documentation next to the SSO backend. Treat this document as the developer handbook built from that contract and the current implementation.